JSERD Monitoring: An External Tool for Editorial Oversight in OJS

Abstract. *JSERD Monitoring is an external tool that monitors editorial work at JSERD in Open Journal Systems (OJS) 3.2.1. It combines OJS reports, API queries, and participant pages. It stores successive states, detects events, computes pending work, and prepares e-mail notifications. Chief editors use a dashboard to inspect evidence and record curation. This paper describes the implemented system, its rules, architecture, and integration decisions for the journal's OJS 3.2.1 installation, including the limits of event detection and notification delivery.*

1. Introduction

The chief editors of the *Journal of Software Engineering Research and Development* (JSERD), a journal sponsored by the Brazilian Computer Society and its Special Committee on Software Engineering [JSERD nd], reported limited engagement with e-mails from *Open Journal Systems* (OJS), which led to manual follow-up. According to their account, the interface separated submissions in progress from processed ones, but did not highlight unanswered invitations, overdue reviews, or other cases that need action. They had to inspect submissions repeatedly to find these cases.

The chief editors wanted to identify these cases and automate messages from templates that fit their needs. OJS 3.2 sends automatic reminders to reviewers only after a due date [Public Knowledge Project ndb]. JSERD also needed notices to several roles, a work list for chief editors, and local follow-up decisions.

JSERD Monitoring[1] meets this need through an external integration with the journal's OJS 3.2.1 installation. Its contribution lies in combining editorial data sources to apply monitoring rules, distinguishing events from persistent conditions, and preserving human curation between collection cycles. The server computes pending work and prepares notifications; the dashboard presents evidence and records chief editors' actions. Editorial decisions remain in OJS.

This paper describes the implemented solution, its integration decisions, and its incremental development from the project requirements. One of JSERD's chief editors tested and validated the solution. The study did not measure effects on messages, monitoring effort, or editorial turnaround times.

2. Editorial Context and Requirements

2.1. Editorial Workflow and Roles

The OJS workflow includes submission, review, copyediting, and production. Review can require several rounds, with different reviewers and requests for author revisions [Public Knowledge Project nda]. A start date and an end date therefore cannot represent a submission. Each reviewer assignment has its own response, deadline, and status within a round.

[1] Source code and supplementary material: https://github.com/srjheam/jserd-monitoring.

In JSERD Monitoring, the *chief editor* uses the dashboard to monitor submissions and record curation, and receives the editorial summary e-mail. The *assigned editor* belongs to the OJS *Associate Editor* group for a submission and receives RN1–RN5 notifications. The *journal editor*, from the *Journal editor* group, receives RN18 when assigned at the *Copyediting* stage. The *reviewer* answers invitations, submits reviews, and receives RN6–RN8 notifications. The *copyeditor*, from the *Copyeditor* group, receives no notification. Only chief editors use the dashboard. The *superuser* administers the server through a separate account without access to the dashboard.

OJS remains the system for editorial decisions, assignments, and reviews. The integration reads its reports, API, and pages with a collection account. The monitoring database stores collected states, findings, notifications, and curation records.

2.2. Events, Findings, and Curation

An *event* records an occurrence that the system detects. Transition events depend on successive states, for example a completion date that changes from empty to filled. Time-based events also depend on the clock. Figure 1 shows this detection without a change in the invitation data.

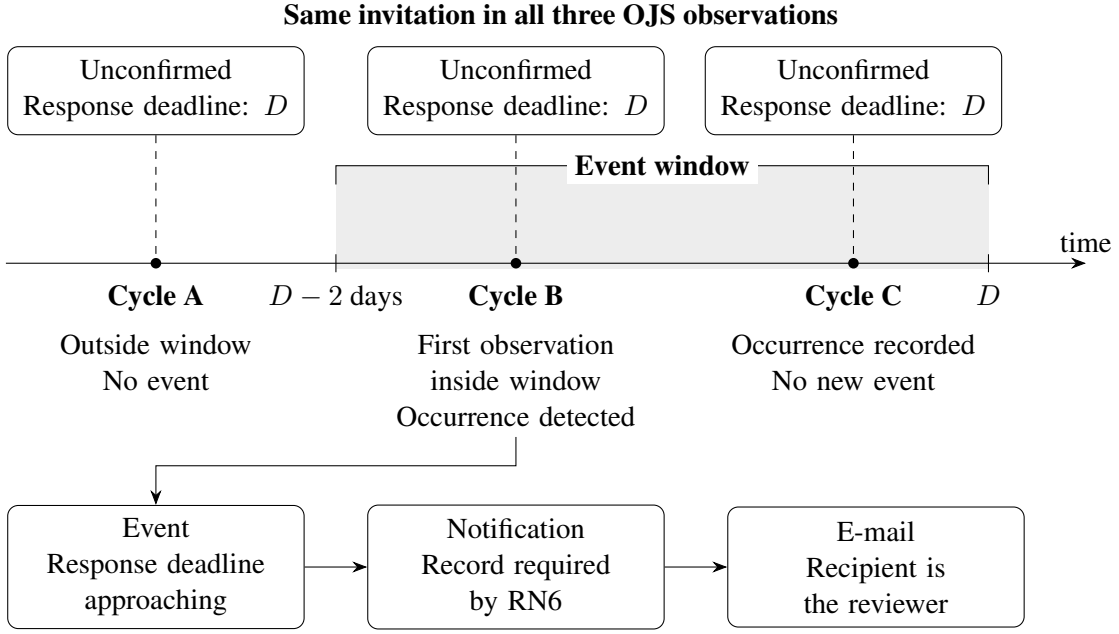


Figure 1. Time-based event detection (RN6). Source: the authors.

A *finding* states that a submission needs the chief editor's attention. It can stay open across several collection cycles without a new event in each cycle. A missing editor is an example of such a condition. A *notification* is a record that a rule specifies, and it produces one *e-mail* per recipient. *Curation* comprises the human actions in the dashboard: notes, conference associations, muting, and marks as up to date.

The monitoring requirements cover notices to editors about invitation responses, missed deadlines, received reviews, completion of all reviews, and signs of a revised manuscript. Reviewers receive notifications in the two-day window before response and

Table 1. Monitoring rules and outputs. RN stands for business rule (Portuguese: *regra de negócio*). AE: assigned editor.

Rule	Condition	Output
RN1	A reviewer answers an invitation (Section 5).	Invitation-answered notice to the AE.
RN2	Invitation unanswered after its response due date.	Notice to the AE.
RN3	Reviewer submits a review.	Notice to the AE.
RN4	All reviews in the round become complete.	Notice to the AE.
RN5	The latest round leaves the revisions-requested or resubmit-for-review state.	Revision-hint notice to the AE.
RN6	Unanswered invitation in the 48 hours before its response due date.	Approaching-deadline notice to the reviewer.
RN7	Incomplete review assignment in the two calendar days before its review due date.	Approaching-deadline notice to the reviewer.
RN8	Incomplete review assignment from the start of its review due date.	Overdue notice to the reviewer.
RN9	Latest open round with fewer than two active reviewers.	Missing-reviewer finding.
RN10	Confirmed, incomplete review from the start of its review due date.	Overdue-review finding.
RN11	Revision still pending more than seven days after the revision-request decision, with no later round.	Missing-revised-manuscript finding.
RN13	Submission has no assigned editor.	Missing-editor finding.
RN15	All reviews of the latest round are complete, with no later decision.	Reviews-complete-without-decision finding.
RN16	Revision no longer pending, more than seven days after the revision-request decision, with no later round.	Revised-manuscript-awaiting-decision finding.
RN17	Submission in the OJS *Submission* stage, before a decision to send it to review.	Desk-decision finding.
RN18	Observed absence of a *Copyeditor* on an active submission in *Copyediting*.	Finding, and a first notice to the assigned *Journal editors*.
RN19	Overdue invitation in the latest round, with fewer than two active reviewers.	Overdue-invitation finding.

review deadlines, and an overdue review notice. Chief editors can consult current findings in the dashboard and in the editorial summary e-mail.

2.3. Rules

Table 1 lists the monitoring rules and their outputs. At the chief editors' request, the system requires two active reviewers per round. It also waits seven days after a revision-request decision before RN11 and RN16 apply. These are configurable journal-specific criteria. Active reviewers are the assignments of the latest round in the assignment that are not declined, cancelled, or overdue for a response.

3. Development and Implemented Solution

For each rule, queries to OJS established which report, API resource, or page provides the data. Development proceeded incrementally through data collection, processing, messaging, and the interface.

3.1. Architecture and Cycle Execution

The Go server [The Go Authors nd] extends PocketBase, which provides authentication, an API, and SQLite persistence [PocketBase nd, SQLite nd]. It serves the Next.js dash-

board as static files [Vercel nd] behind a Caddy reverse proxy [Caddy nd]. Figure 2 shows the components and the data exchanges. The SMTP server is an external dependency.

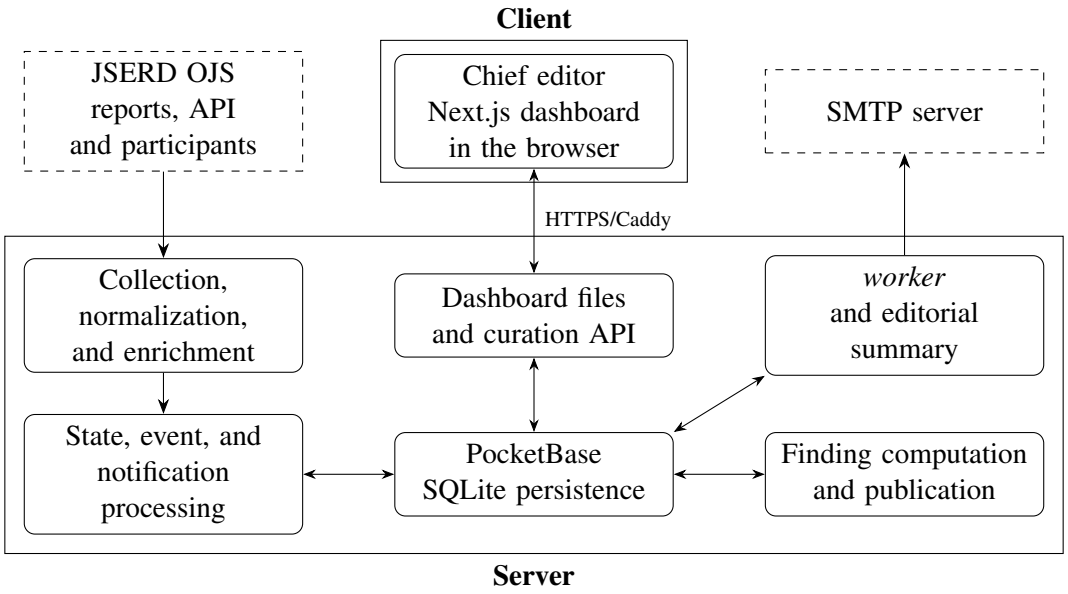


Figure 2. Architecture and data exchanges. Solid outlines enclose the client and server. Dashed boxes denote external dependencies. Source: the authors.

A *cycle* is the recurring unit of monitoring. By default, a cycle runs every hour, the e-mail worker every minute, and the editorial summary on Mondays at 09:00, São Paulo time. The superuser can also request a manual cycle. The coordinator controls the cycle sequence and rejects a second run while a cycle is in progress in the same process. The design assumes one server process. Collection signs in, downloads the article and review reports, and reads the submissions API. It records the source and outcome of each download, including failures.

Normalization converts the data into a common internal model. *Enrichment* adds details from individual submission queries. Processing stores the normalized state, compares it with the previous state, detects events, and matches notification rules. The server then computes findings, updates the dashboard, and checks the summary schedule.

Enrichment runs twice in each cycle. The first pass runs before the server saves the snapshot, because RN2, RN5, RN7, RN8, and RN18 need live OJS status to create events. The second pass runs before finding computation. It reloads the saved snapshot and queries invitation status, due-date status, and the active reviewers of the latest round again. A failed query stops the cycle. Committed transactions remain, and the next cycle queries OJS again. If the second pass fails, the snapshot and events remain, and the dashboard keeps its previous findings until the next successful cycle.

3.2. Combining Sources and Evidence

Collection reads three OJS sources. The *ArticleReportPlugin*, *ReviewReportPlugin*, and *ViewReportPlugin* reports provide submissions, assignments, and metrics in batches. The REST API lists submissions with a summary schema, and an individual query returns stage, rounds, and assignments [Public Knowledge Project ndc]. The stage participant

page lists the people assigned to each stage. The collection account needs access to all three sources. The server queries details only for submissions that are candidates for rules that need them.

The sources identify people differently. The Reviews report gives each reviewer a username and an e-mail address, and reviewer notices go to that address. The participant page gives only the names of assigned editors. The system matches these names with the editor e-mail addresses in the article report. The API detail gives the status and due date of each assignment, but no reviewer identity. The system therefore applies API status to all report rows that share a round, or a round and a review due date.

For RN5, an author revision is pending when the latest round is in the revisions-requested or resubmit-for-review state [Public Knowledge Project 2020]. The event occurs when a cycle observes the round in another state after the previous cycle observed it as pending. The new state can be a submitted revision, but also an accept or decline decision. RN5 is therefore an indicator that can produce false positives, and it does not confirm a file or its author. RN16 uses the same round-state proxy.

RN18 distinguishes an assigned *Copyeditor*, an observed absence, and unknown evidence. Only an observed absence produces a finding. Evidence is unknown, for example, when the article report shows a submission in *Copyediting* but the individual query no longer does.

3.3. Successive States and Duplicate Prevention

The system stores a *snapshot*, a copy of the normalized state, for each processed cycle. Snapshot comparison detects transitions without changes to OJS. The first snapshot is the baseline for transitions, but it can already produce time-based and missing-copyeditor events.

Each event receives a key computed from its type, entity, transition, and occurrence. For time-based events, the occurrence includes the deadline, so a deadline extension produces a new event. The database enforces key uniqueness, so reprocessing an occurrence creates no new record, as in the *Idempotent Receiver* pattern [Hohpe and Woolf nd].

One transaction records the cycle, the snapshot, and the events. A second transaction creates the notification and one e-mail record per recipient before any SMTP communication. Only this second step applies the *Transactional Outbox* pattern [Richardson nd]. Uniqueness constraints and the worker reservation give three guarantees: at most one event per key, one notification per event and rule, and one send attempt per e-mail.

3.4. Notifications and Editorial Summary

The e-mail worker reserves pending e-mails and records the outcome of each send attempt. The persisted reservation prevents two workers from processing the same e-mail. The first notification to a person also produces a welcome e-mail.

Sending uses SMTP [Klensin 2008] or the default simulated transport, which writes e-mails to the server output. The templates are in English, have plain-text and HTML versions, and are part of the code. A failure to create a notification does not stop finding computation, and later cycles resume event-rule matches without a notification.

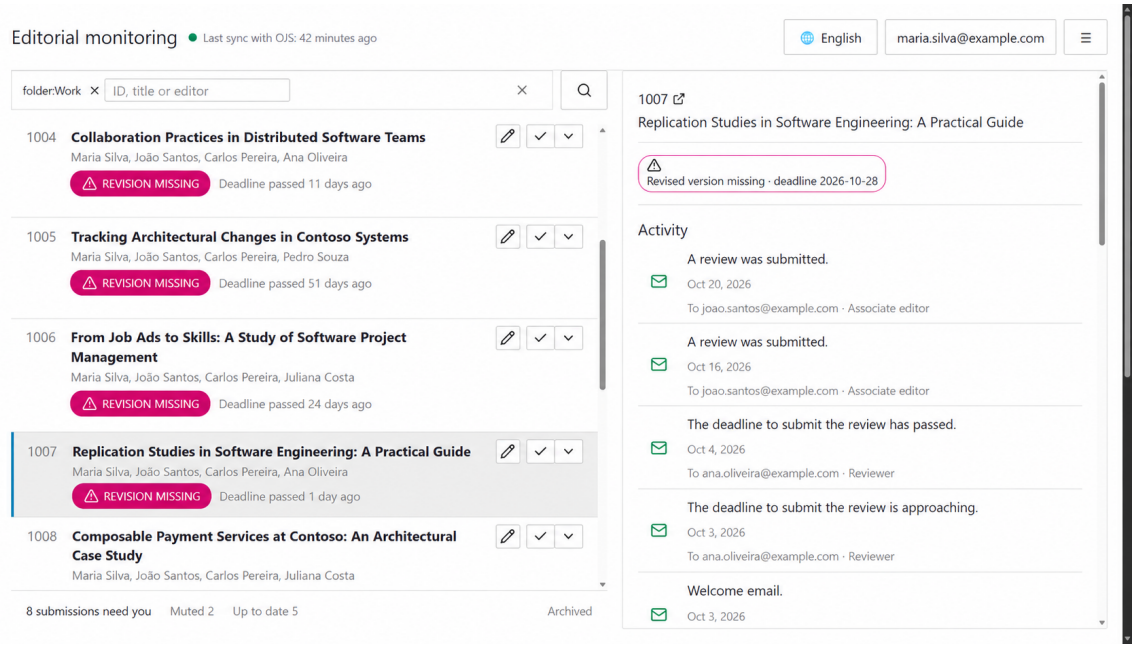
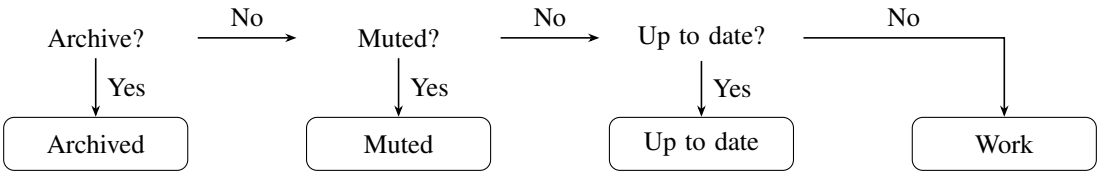


Figure 3. Dashboard with the work list and a selected submission. All data are fictitious. Source: the authors.



Archive: final OJS state, or an active *Copyediting* submission with an assigned *Copyeditor*. **Up to date:** no findings, or a manual mark that is still valid. The first condition that holds determines the queue.

Figure 4. Dashboard queue precedence. Arrows show the classification order. Source: the authors.

3.5. Dashboard and Preservation of Curation

The dashboard lists submissions with their findings and the supporting data. The work list orders submissions by their most severe finding. The detail view shows OJS evidence, send history, and curation notes, with a link to the submission in OJS. The dashboard is available in Portuguese and English. Figure 3 shows the work list and the send history of one submission with fictitious data.

Each submission belongs to exactly one queue, assigned in this order: archived, muted, up to date, and work (Figure 4). Final outcomes, such as publication, rejection, and withdrawal, depend on the current OJS state.

When a chief editor marks a submission as up to date, the system saves a digest of the finding types present at that moment, without deadlines or counts. The mark expires when the set of types changes or when the return date that the chief editor chose arrives.

Muting removes a submission from the work list when a trusted editor handles it. Conference associations exist only in JSERD Monitoring. A trusted editor handles

the submissions of a conference, so selecting or changing a conference also mutes the submission, and removing the association unmutes it. The chief editor can still unmute a submission and keep its conference. The system still collects and evaluates muted submissions, but their findings do not return them to the work list. Muting also removes a submission from the editorial summary, but editors and reviewers still receive event notifications.

Curation data remain separate from data that the system recomputes. A new cycle can update the state and findings of a submission, but notes are immutable. Access follows the same split. Chief editors change curation data, and only the superuser administers the server.

3.6. Access, Personal Data, and Availability

The dashboard requires chief editor authentication and shows the same journal view to every account. The OJS and SMTP credentials reach the server as environment variables from a file on the host.

The database contains submission and assignment data, contact data for authors, editors, and reviewers, and curation records. Following the necessity principle in Art. 6 of the LGPD [Brasil 2018], snapshots do not keep review comments, recommendations, or reviewer profile columns, which no rule reads. The raw report files keep every column, including review comments. Retention keeps raw files and snapshot contents for the 240 most recent cycles, about ten days at the default schedule. A daily off-site backup contains the PocketBase databases and the latest state, but not the full history of raw files and states.

4. Related Work

Journal systems offer part of this monitoring. OJS 3.2 sends reviewer reminders after due dates [Public Knowledge Project ndb], and it e-mails stage editors when a reviewer accepts or declines an invitation [Public Knowledge Project 2020]. JSERD Monitoring adds rules that combine several sources, findings for chief editors, and local curation, without changes to OJS.

Lopes and Oliveira describe a framework that queries sources periodically, compares the results with internal storage, and triggers template-defined operations, including e-mails, for unrecorded content [Lopes and Oliveira 2015]. It detects new content through identifiers or hashes. JSERD Monitoring compares normalized states to detect editorial transitions, and other rules consider elapsed time and persistent conditions.

Labio and Garcia-Molina [Labio and Garcia-Molina 1996] study snapshot comparison to infer changes. Duplicate prevention relates to the *Idempotent Receiver* pattern [Hohpe and Woolf nd], and the e-mail records relate to the *Transactional Outbox* pattern [Richardson nd].

5. Discussion and Limitations

The article report and the participant page identify editors by name, without a stable identifier. Name normalization merges people with the same name, also across the editor and reviewer roles. The API detail has no reviewer identity. The Reviews report keeps declined and cancelled assignments, and a decline also fills the confirmation date [Public Knowledge Project 2020].

Periodic collection delays change detection. A cycle represents the data that its queries obtained. Report and detail queries do not form a single OJS transaction. Transitions before observation began have no previous state, and intermediate changes between cycles can go unobserved.

The integration depends on permissions, report plugins, the structure of participant pages, and OJS status codes [Public Knowledge Project 2020], and not all of them have a stable interface. Another OJS version or journal requires a check of these sources and of local criteria.

A prepared notification does not guarantee delivery. Missing addresses prevent sending. The worker does not retry failed attempts. Ordering is not guaranteed, and SMTP acceptance does not prove arrival or reading. The design therefore does not give the delivery guarantees of the *Transactional Outbox* pattern.

6. Conclusion

JSERD Monitoring is an external monitoring layer for the OJS installation of JSERD, while editorial decisions remain in OJS. The server combines reports and individual queries, stores successive states, recognizes events, computes findings, and prepares notifications. The implementation combines sources according to the data each rule requires, separates events from persistent conditions, and preserves human records between collection cycles. Adapting it to another journal requires reviewing local criteria, permissions, and OJS interface contracts.

References

Brasil (2018). Lei n. 13.709, de 14 de agosto de 2018: Lei geral de proteção de dados pessoais (LGPD). Arts. 6, 7, 15, 16, and 46. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Accessed: 23 October 2026.

Caddy (n.d.). *Reverse proxy quick-start*. https://caddyserver.com/docs/quick-starts/reverse-proxy. Accessed: 23 October 2026.

Hohpe, G. and Woolf, B. (n.d.). Idempotent receiver. Enterprise Integration Patterns. https://www.enterpriseintegrationpatterns.com/patterns/messaging/IdempotentReceiver.html. Accessed: 25 October 2026.

JSERD (n.d.). About the journal. https://journals-sol.sbc.org.br/index.php/jserd/about. Accessed: 20 October 2026.

Klensin, J. (2008). Simple mail transfer protocol. RFC 5321, Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc5321. Accessed: 21 October 2026.

Labio, W. J. and Garcia-Molina, H. (1996). Efficient snapshot differential algorithms for data warehousing. In *Proceedings of the 22nd International Conference on Very Large Data Bases*, pages 63–74. Morgan Kaufmann. https://www.vldb.org/conf/1996/P063.PDF. Accessed: 25 October 2026.

Lopes, P. and Oliveira, J. L. (2015). An automated real-time integration and interoperability framework for bioinformatics. *BMC Bioinformatics*, 16:328. https://doi.org/10.1186/s12859-015-0761-3. Accessed: 26 October 2026.

PocketBase (n.d.). *Introduction*. https://pocketbase.io/docs/. Accessed: 21 October 2026.

Public Knowledge Project (2020). PKP Web Application Library, version 3.2.1-2. Source code. https://github.com/pkp/pkp-lib/tree/3_2_1-2. Accessed: 4 November 2026.

Public Knowledge Project (n.d.a). *Learning OJS 3.2: A Visual Guide to Open Journal Systems*. https://docs.pkp.sfu.ca/learning-ojs/3.2/en/. Accessed: 25 September 2026.

Public Knowledge Project (n.d.b). *Learning OJS 3.2: Workflow Settings*. https://docs.pkp.sfu.ca/learning-ojs/3.2/en/settings-workflow. Accessed: 23 October 2026.

Public Knowledge Project (n.d.c). *Open Journal Systems: API Reference, version 3.2.x*. Official specification for version 3.2.1-2. https://github.com/pkp/ojs/blob/3_2_1-2/docs/dev/swagger-source.json. Accessed: 21 October 2026.

Richardson, C. (n.d.). Pattern: Transactional outbox. https://microservices.io/patterns/data/transactional-outbox.html. Accessed: 25 October 2026.

SQLite (n.d.). *About SQLite*. https://www.sqlite.org/about.html. Accessed: 23 October 2026.

The Go Authors (n.d.). *Documentation*. https://go.dev/doc/. Accessed: 23 October 2026.

Vercel (n.d.). *How to create a static export of your Next.js application*. https://nextjs.org/docs/app/guides/static-exports. Accessed: 21 October 2026.